Shared Memory Parallelism Report

November 2020

Contents

1 README **1**
1.1 parallel.c 1
1.2 sequential.c 1
1.3 Output 1

2 Implementation **2**
2.1 Array Generation and Input 2
2.2 Computation 2
2.3 Avoiding Race Conditions 2
2.4 Precision 3

3 Correctness Testing **4**

4 Scalability Investigation **5**
4.1 Efficiency 18
4.2 Speedup 20
4.3 DELETE BEFORE SUBMISSION 20

A Code **20**

1 README

1.1 parallel.c

The compiled C file (compile with the command `gcc parallel.c -lpthread -lrt`) can be run with or without the following arguments, in the stated order:

1. Dimension of input array. For example, `5` for a 5x5 array. Must be > 2 to have mutable elements Default is `18`.

2. Precision. For example, `0.1`. Default is `0.01`.

3. Number of threads to run. Any number of threads up to and including the number of mutable array elements will run, but note the maximum number of CPU cores on one Balena node (16). Default is `16`.

4. Custom array; input the name of the text file (`.txt`) containing the array as a command line argument to import the array. Array must match input dimension. Separate each value with one space only. Default is an array of random integers between 0 and the array dimension.

Commands *must* be entered in this order, and with no gaps. For example, `./parallel.out 20 0.1` is permissible for a `20x20` array and a precision of `0.1`, but `./parallel.out 20 10` will not run a `20x20` array with `10` threads.

1.2 sequential.c

The compiled C file (compile with the command `gcc sequential.c -lrt`) can be run using the same arguments as above and with the same caveats, with the exception that 'Number of Threads' may not be specified (as the program runs serially using a single thread of execution), and hence argument 4 (filename) should be entered as argument 3 if it is to be used. Identical defaults are used.

1.3 Output

By default, the provided files will output their results in the form of a labelled file: either `parallel-results-$dimension-$precision-$threadCount.txt` for `parallel.c`, or `sequential-results-$dimension-$precision.txt` for `sequential.c`. Here `$dimension` refers to the array dimension, `$precision` to the input precision, and `$threadCount` to the number of threads being used, either as defined via the command line input outlined above, or set to their default values of `18`, `0.01`, and `16` respectively.

2 Implementation

2.1 Array Generation and Input

The default array generated will be a random array of integers between 0 and the array dimension (either the input array dimension or the default array dimension of 18. To use custom arrays, see README.

The default array is generated by producing a random integer between 0 and `RAND_MAX` (a system-dependent value guaranteed to be at least 32767) and computing this value modulo the array dimension. Provided a large enough difference between `RAND_MAX` and the array dimension, this produces a pseudo-random integer between 0 and the array dimension. To ensure both the parallel and the sequential programs generate the same array, the random seed in both programs is set to the same value (in the case of the scalability investigation herein, it is set to 1), and can be altered in the code for experimental purposes.

2.2 Computation

This implementation of the relaxation technique uses `pthreads` to produce a solution, using a `do-while` loop that continues until no elements in the array change by more than the defined precision in the most recent. In each iteration, all threads operate on at least $(arraySize - 2)^2 \ /threadCount$ mutable array elements, with the 'remaining' elements (if the number of mutable elements, $arraySize - 2$, is not divisible by the number of threads) operated on by one thread each. For example, where a 20x20 array was being operated on by 16 threads, there would be 324 (18×18) mutable elements, so 12 threads would operate on 20 (324/16) array elements each and 4 (324 % 16) threads would operate on 21 each. This ensures that work is as evenly divided amongst the threads as possible, maximising the gains from parallelising operations and minimising the time threads (and hence processors) spend idle.

2.3 Avoiding Race Conditions

In order to avoid a race condition whereby threads operating on values from different iterations, two different checks were required. Firstly, two separate arrays are used to store values: one to hold the values produced from the last iteration, and one to hold those produced by threads in the current iteration. Threads then read values from the former array when calculating the

mean of an element, before writing to latter; at the end of the iteration all threads (including main) call `pthread_barrier_wait()` on the same barrier, a barrier that was initialised before the threads were created. This ensures that all threads have finished iterating their values, and the iterable array is 'finalised' for this iteration.

Each thread other than the main control thread (that runs `main()`) then calls `pthread_barrier_wait()` again - this allows for the control thread to run flag updates and to check if the program is finished before the other threads resume, while the use of barriers allows for the re-use of threads, reducing the overhead of thread creation and termination. Additionally, the two array pointers are swapped using a temporary 2D array pointer as an intermediary, providing a new 'empty' array with the same boundary values to iterate on, and an up-to-date array to read values to average from. This ensures that threads cannot 'run ahead' by multiple iterations, or operate on an array where some values have been updated and others have not. This process could have further been improved by the implementation of a 'thread pool', but unfortunately this was not completed.

2.4 Precision

Iteration continues until all elements in the array, when iterated again, change by less than the given precision. This process is also parallelised. A Boolean flag, set to `true` at the beginning of each iteration. Each time a thread operates on an element a precision check is then conducted: if the absolute value of the difference between the former value and the new value of the element is greater than the precision, the flag is changed to false, and remains false for the rest of the iteration — else it remains true. The program then continues iterating until the flag is set to true even once all threads have called `pthread_barrier_wait()`, at which points the program outputs the result and exits.

3 Correctness Testing

In order to conduct correctness testing the results produced by the parallel program must be compared to a 'ground truth', in this case produced by a sequential program utilising the same algorithm. It is assumed that this sequential program produced 'correct' outputs: these outputs were manually verified for several small arrays and large precisions, and for several larger arrays was also verified against the output arrays of several other students.

In order to automate correctness testing for a large number of arrays and a variety of precisions, a bash script was produced that could schedule multiple jobs at once via a loop that iterated through various array sizes, precisions, and additionally several different thread quantities for the parallel version (in an attempt to expose any rare race conditions). Both the parallel and sequential code was then altered slightly to write the output array to a file, uniquely named for that particular run — for example, `sequential-results-1000-0.01` would identify the output of the sequential program run on an array size of 1000 and to a precision of 0.01. The content of the arrays itself was manually altered between runs of the script - for example, one set of arrays consisted of randomly allocated 1s and 0s, whilst another consisted of a set boundary value whilst the interior of the array was populated with 0.0s.

The bash `cmp` command was then used to compare the contents of the files containing the outputs of the sequential program with the parallel program for a number of different thread counts. On all tests run, on arrays from size 3–10,000, the contents of the files were identical. Hence it can be concluded that the parallel implementation provided is 'correct'.

4 Scalability Investigation

In order to assess how the parallel implementation scaled, it was necessary to develop a script that could automate running large numbers of jobs. These jobs would be performed using various array dimensions, precisions, and numbers of threads, and collect statistics including running time, CPU utilisation, and number of iterations to complete. This script ran 'standard' versions of the included code (no compile-time optimisations).

The primary array type ('type' referring to the content of the array) used for these tests was a random array of values between 0 and the array dimension, as was mentioned above as the current default setting for the program, and in particular an examination of this array type to a precision of 0.01. This was due to a relatively strong correlation between array dimension and the number of iterations it took to produce a solution; a random seed of 1 was selected for this reason.

A variety of array types were experimented with, but for many of the metrics detailed in this report produced similar results, if at a different scale. For example, initial tests were conducted with an array with a boundary of 1.0, and otherwise filled with 0.0s. This array could scale to just over `7000x7000` on a precision of 0.01 before hitting the Balena time cap of 15 minutes, though this array also scaled in manner that was not generalisable, as iterating the array simple resulted in values that got smaller as they moved towards the centre, and so after a certain size a larger array simply resulted in a larger 'middle' section of 0.0s. In particular, the use of an array of random 1.0s and 0.0s displayed some interesting properties. Where array types other than the 'default' have been used it has been noted; in particular the above described array is listed as the '1s and 0s' array type.

The first (and simplest) metric to measure was simple program running time for arrays of different dimensions. Across all thread quantities measured (see Figure 1) running time (measured using the `perf` tool) increasing broadly exponentially as array size increased. This held as long as increased array size increased with the number of iterations it took to process. This is notable number of iterations increased linearly with array size (see Figure 2), though this occurred in two notable segments — between array dimensions of 0 and 250 and 250 upwards. This was linear scaling an effect of the chosen random seed, as it was not demonstrated when using alternate seeds. The rate of linear increase had a direct effect on the rate of exponential increase — as can be seen in Figure 3, an increase in array size from 0x0 to 250x250 displays

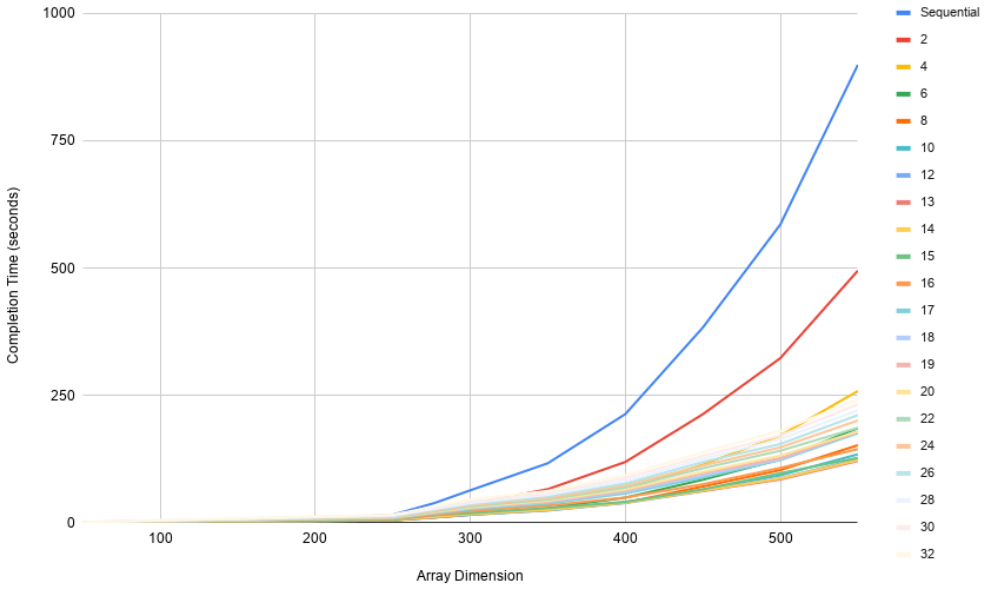

Figure 1: Program total running time vs array dimension for 1–32 threads

a similar exponential increase, but at a far slower rate than is demonstrated from 250x250 to 550x550, when rate of linear increase in iteration number is far greater

The compounded effects of an increase in the number of elements each thread has to process and the number of iterations taken to process the array, the former increasing quadratically with array dimension and the latter linearly, likely resulted in this scaling. Additionally, the fact that the range of values in the array increases with array size also likely had a significant effect, as this exponential behaviours is not seen in other array types, such as when arrays are formed from a random series of 1s and 0s (see Figure 4).

Reference data on iteration number vs array size when I talk about it above

Optimal thread count was largely dependent on dimension. A trend displayed across all arrays types tested was that as array size increased the optimal thread count trended upwards (though at varying rates depending on the array type) towards a limit of at least 15 on conducted tests. As

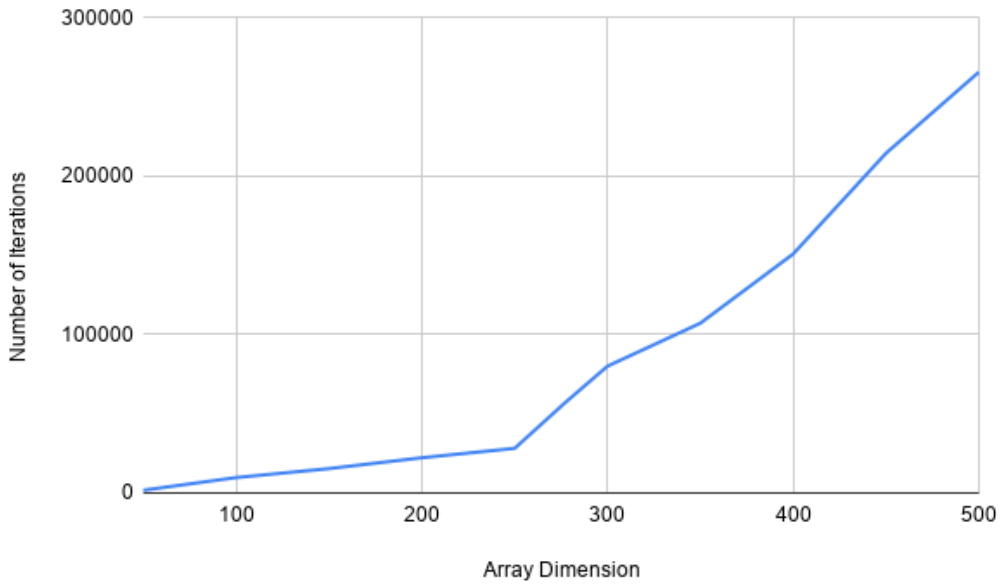

Figure 2: Number of iterations to solve array to a precision of 0.01 vs array dimension

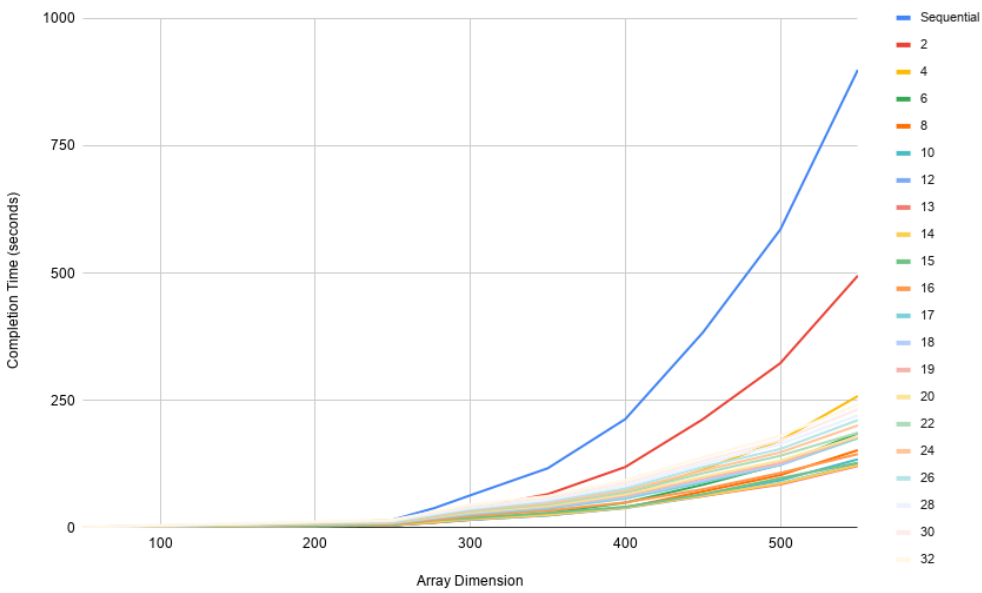

Figure 3: Program total running time vs array dimension up to a dimension of 250

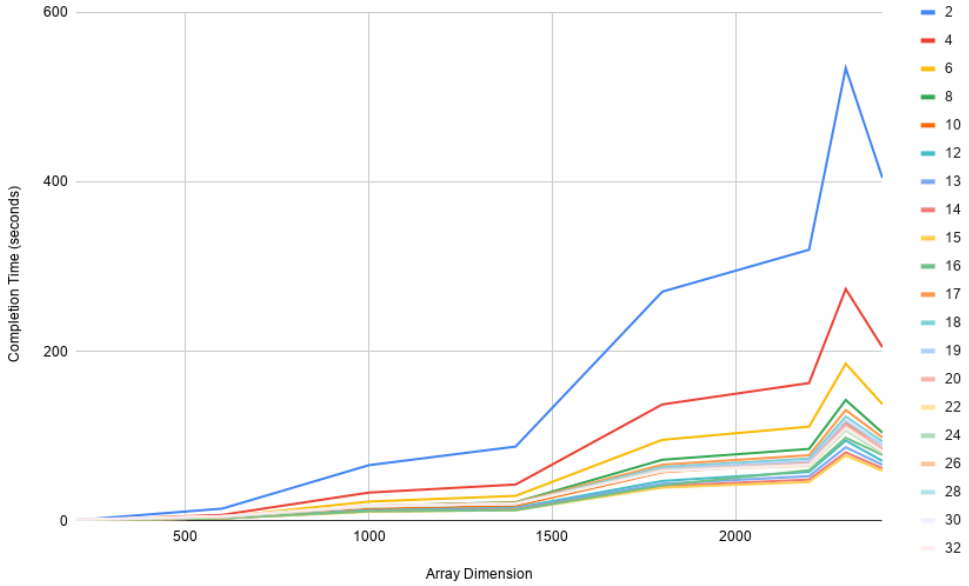

Figure 4: Program total running time vs array dimension for 1–32 threads - 1s and 0s array

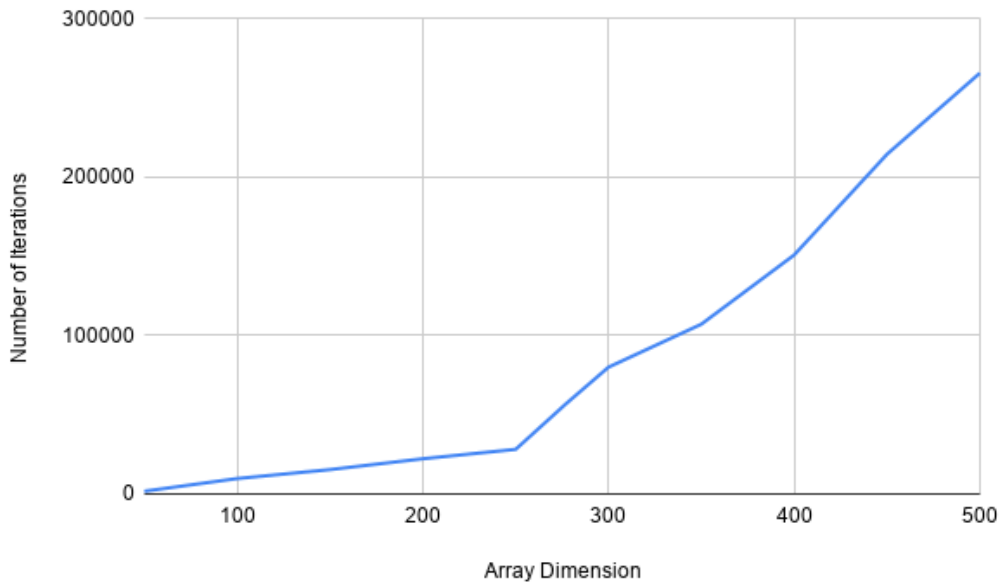

Figure 5: Number of iterations to solve array to a precision of 0.01 vs array dimension

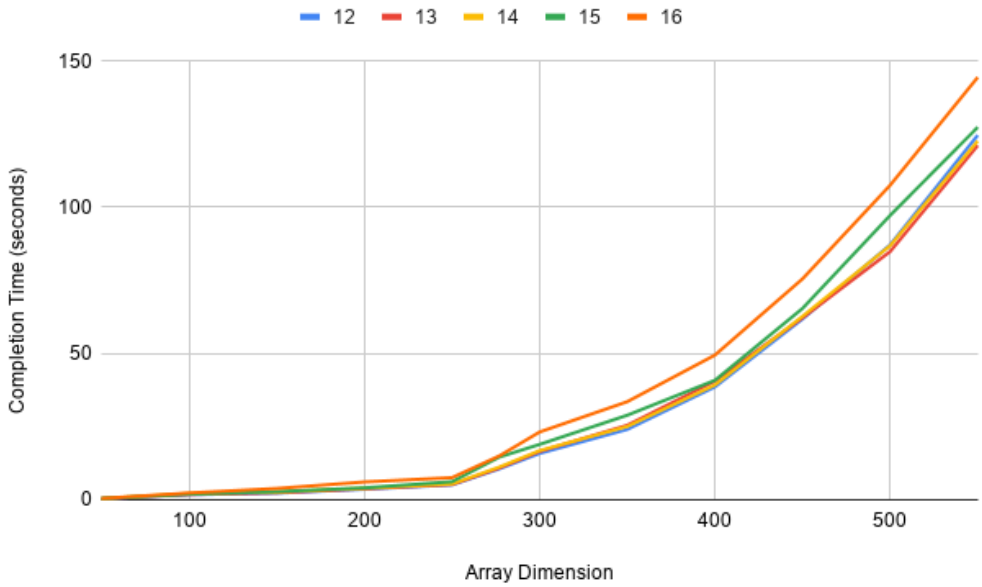

Figure 6: Program running time vs array dimension for 12–16 threads only

can be seen in both Figure 1 and more clearly in 16 Threads)fig:Time vs Dimension (12–16 Threads)Figure 6, for the smaller array sizes tested (those that could run sequentially within 15 minutes) the optimal thread count was 13, yet when tested at array sizes of 800x800 upwards (Figure 7) 15 threads became optimal. With alternate array types, 15 threads appeared as the optimal count with arrays upwards of 600x600. The increase in optimal thread count as array size scaled is even more transparent in

reference speedup graphs here

Reference raw data at the end of the above paragraph

It's possible that at larger array sizes, un-testable within a 15 minute time-limit with the array types developed, 16 threads would become the optimal count, as this is the number of processors present in the node. However it's also possible that 15 is in fact the highest optimal count present, with increased communication between the processor cores producing an overhead at 16 cores that only scales upwards with greater workloads. An additional explanation for a lower maximum optimal thread count may be control threads used by the Balena infrastructure occasionally preempting regular process-

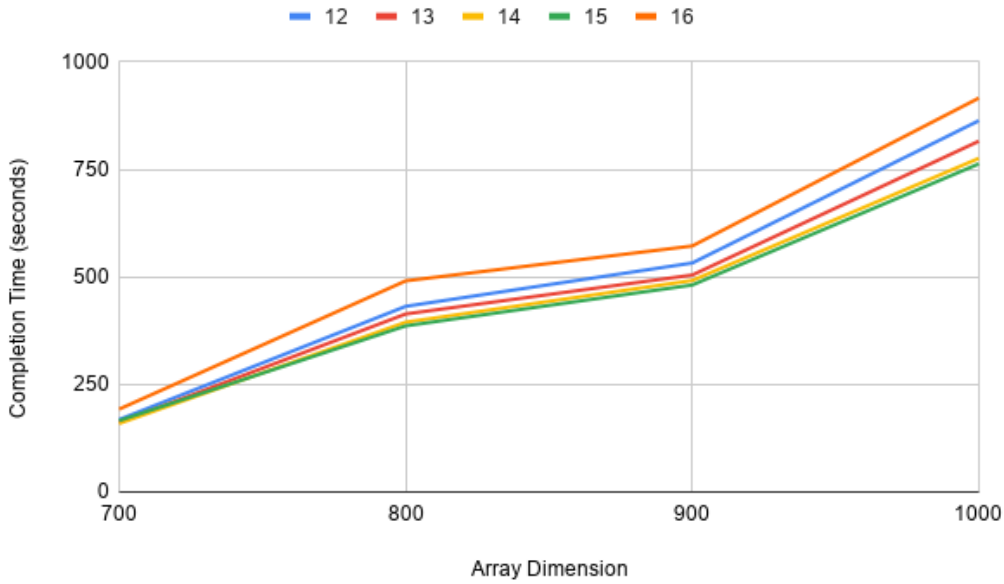

Figure 7: Program total running time vs array dimension for larger array sizes

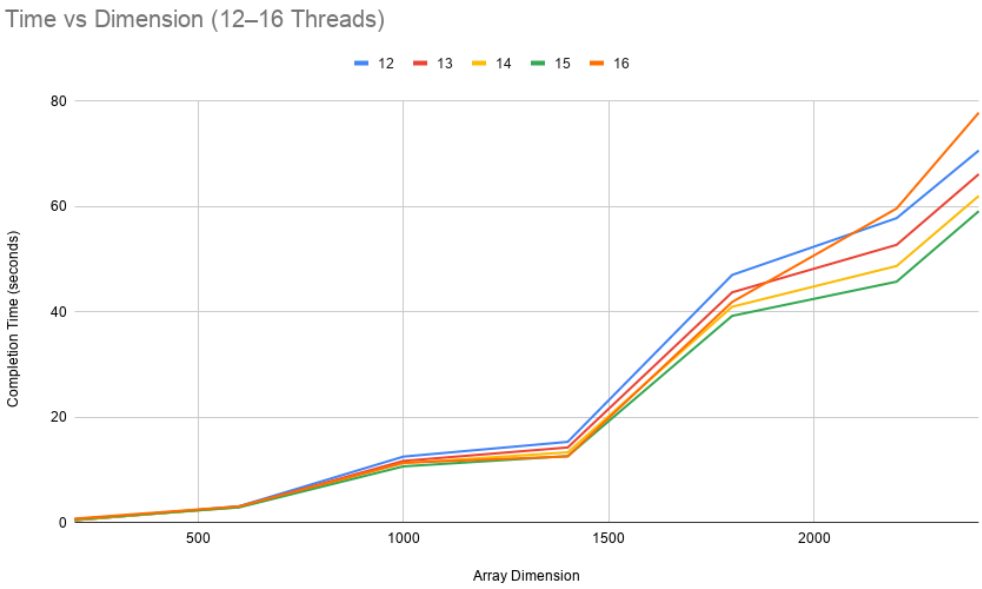

Figure 8: Program total running time vs array dimension for 12–16 threads — 1s and 0s array

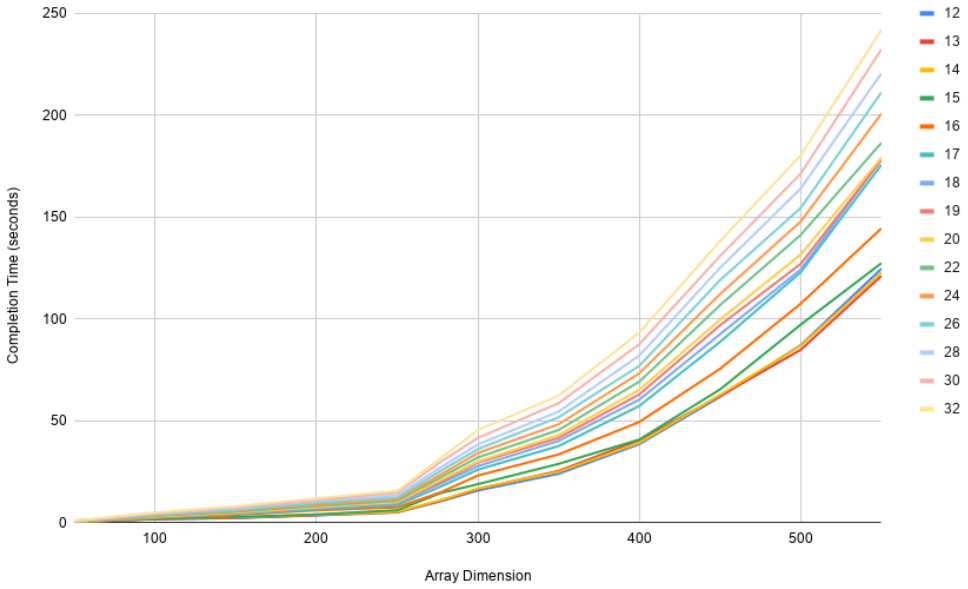

Figure 9: Program total running time vs array dimension (12–32 Threads)

ing, resulting in the necessity of scheduling at a thread count of less than 16.

When the number of threads was scaled up beyond 16, the running time dropped again, due to the overhead of scheduling multiple threads on each processor core. The slowest average running time was for 32 threads, with the running time gradually increasing as the number of threads increased past 16 and a notable jump from 16 to 17 threads (see 32 Threads)fig:Time vs Dimension (12–32 Threads)Figure 9). This is simply explainable by the increase in scheduling time that is required by greater thread counts. However, for alternate array types a flipped trend was displayed, with 17 threads actually running slowest of all (see 32 Threads — 1s and 0s arrayfig:14–32 Threads — 1s and 0s arrayFigure 10) and 32 threads running the fastest of all thread counts greater than 16, with a final running time for 32 threads only slightly slower than that for 16 threads.

COULD BE THAT 16 IS BEST WHEN SCALED UP EVEN MORE, MAYBE RUN SOME WITHOUT CORRESPONDING SEQUENTIAL RUNS?

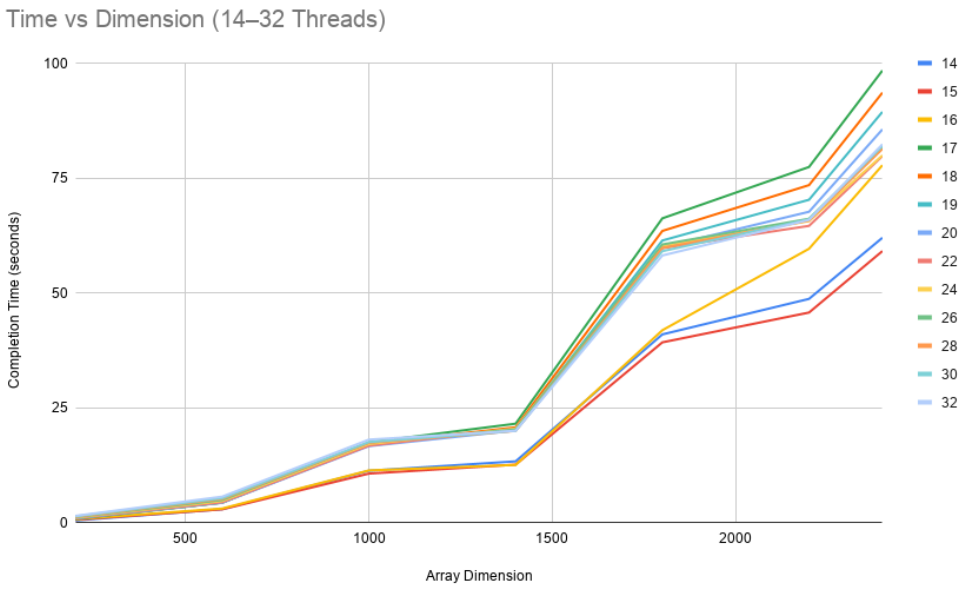

Figure 10: Program running time vs array dimension for 14–32 threads — 1s and 0s array

One interesting, and unexpected result of the scalability investigation was the running time per *iteration*. As can be seen in both main array types tested (see Figure 11 and Figure 12) for not only 15 threads but all threads the time per iteration grew as array size increased; indeed for the 1s and 0s array type the rate of increase even grew as well. This was a surprising result, as it was to be expected that the time per iteration would stay largely stable, with perhaps a slight decrease as overheads such as thread management and scheduling became less significant as a proportion of running time. The most likely explanation for in the experiments used as iteration count grows, so does array size, meaning that the number of elements to process in each iteration grew alongside the number of total iterations the program ran. Further investigation could be conducted in the future to confirm this by exploring how arrays where total iterations dropped as array size increased behaved.

Another surprising result was produced by investigation into the amount of time CPUs spent idle. Using the `perf` tool, it was discovered that for a static array size, the proportion of CPU cycles spent idle increased alongside thread count (see Figure 13). In particular there was marked jump from 1 to

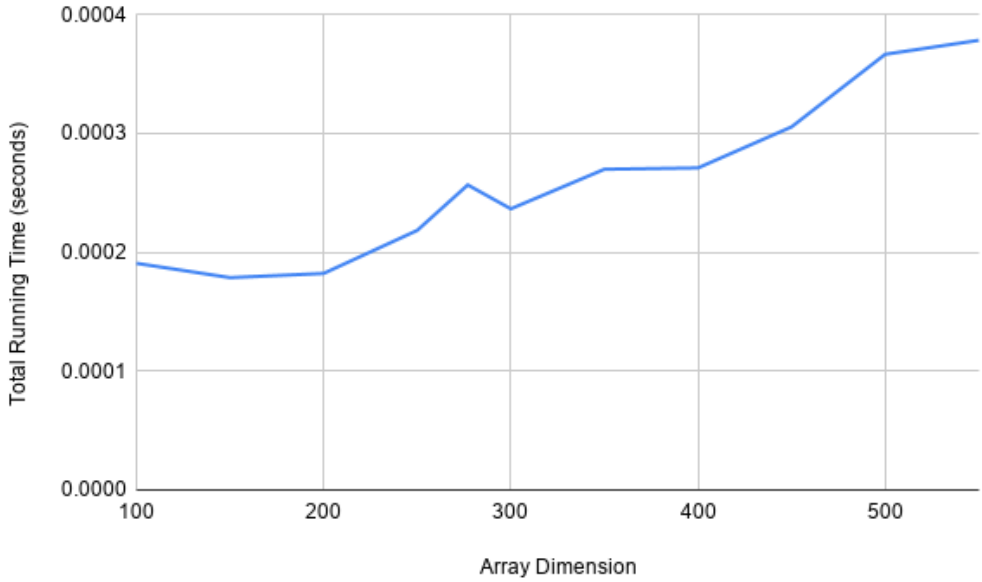

Figure 11: Time per iteration vs array dimension

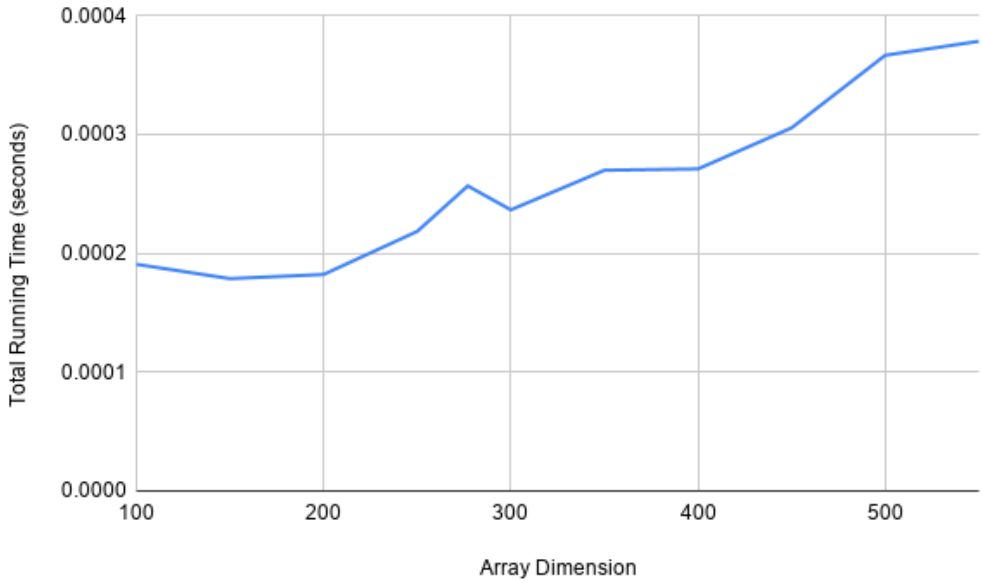

Figure 12: Time per iteration vs array dimension — 1s and 0s array

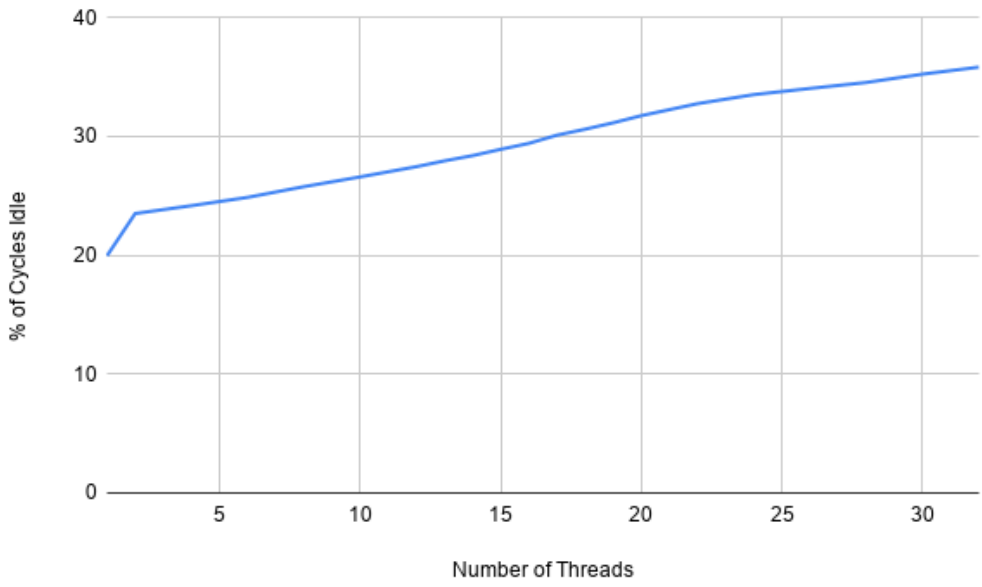

Figure 13: % of CPU cycles spent idle for constant array size (550x550)

2 threads from 19.95% idle to 23.50% (due to the introduction of pthreads and their associated management), but the further stable rise is likely due to the average increase in the time threads spend blocked by a barrier as thread count increases. The inverse relationship is shown with increasing array size for a static thread count (see Figure 15), tending towards around 20–25% of CPU time spent idle, as a result of the relative reduction in thread management. As each thread has more elements to operate on, time spent blocked and in serial parts of the program reduces relative to the time spent processing elements in parallel.

Talk about the effect on speedup?

Maybe move this section to where we talk about speedup

Include more RAW DATA in paragraphs above Produce more data

Despite this large amount of time spent idle, average CPU utilisation (i.e. the average number of CPUs in use) was relatively high across all runs (see Figure 16 and Figure 17). Initially rising almost precisely in line with thread count, CPU utilisation was nevertheless slightly less (0.1–0.5 less) than the thread count for the 1s and 0s array, likely due to processor communication overheads, and considerably less (almost half) for the default array. The

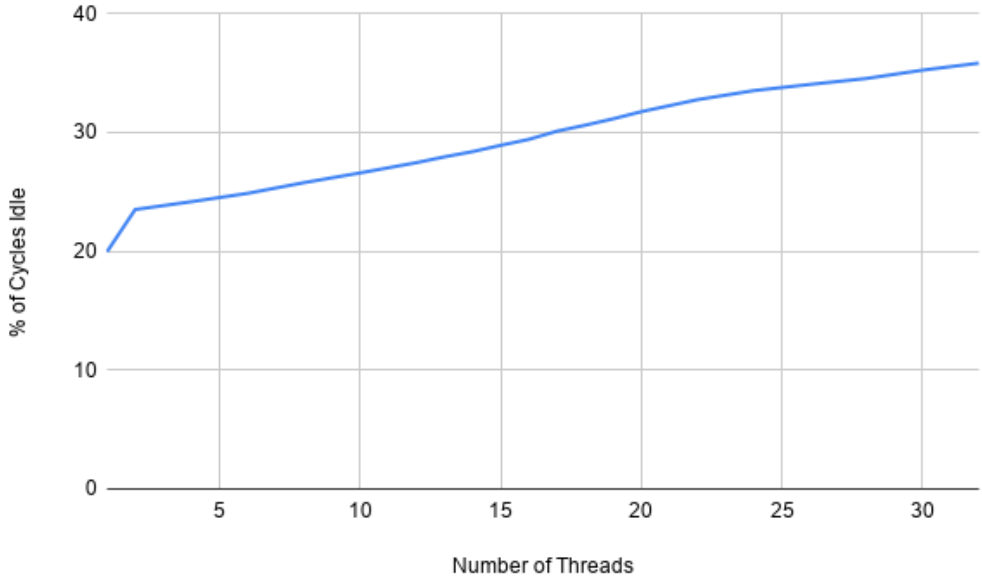

Figure 14: % of CPU cycles spent idle (2200x2200) — 1s and 0s array

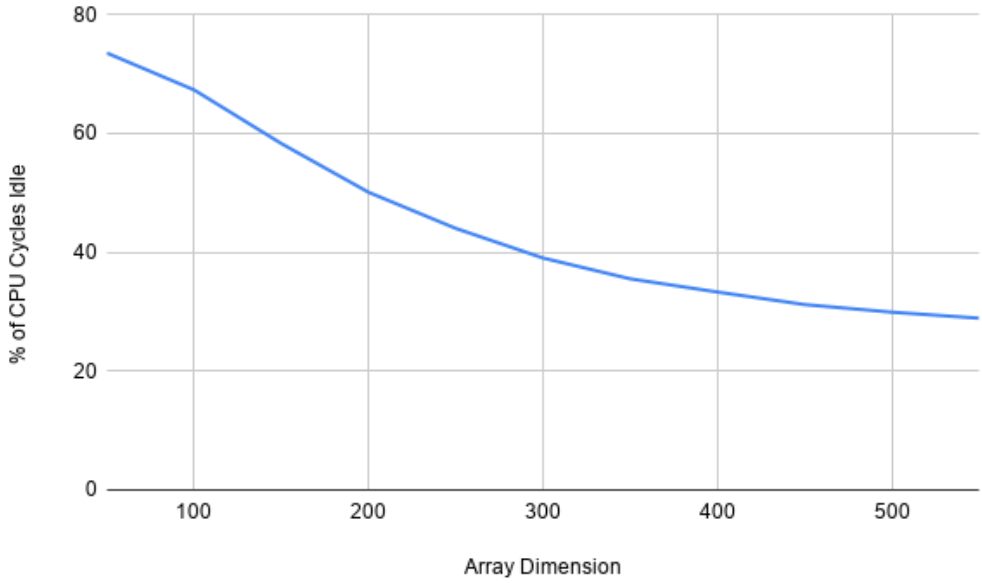

Figure 15: % of CPU cycles spent idle for constant thread count (15 threads)

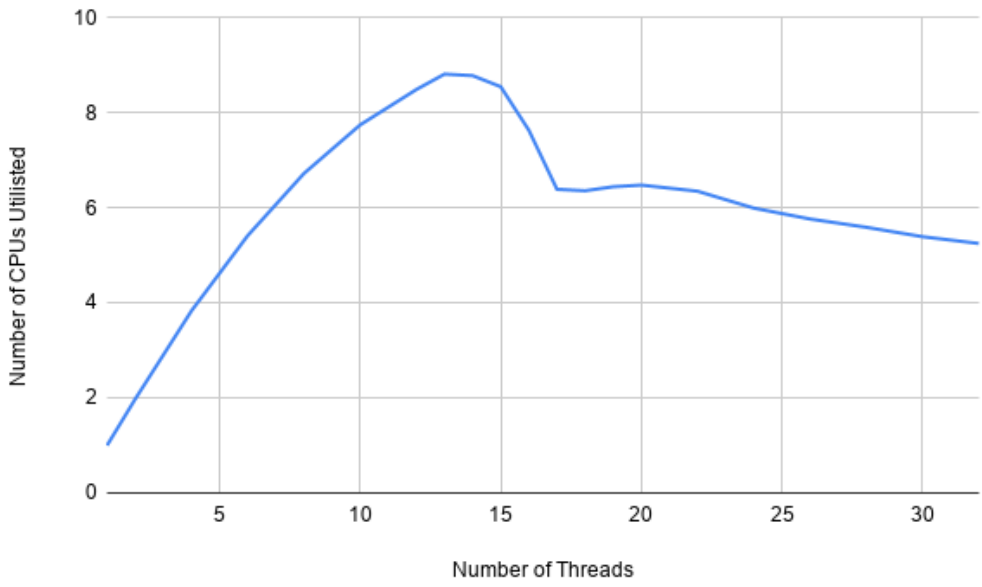

Figure 16: Average Number of CPUs Utilised (550x550)

highest CPU utilisation (14.219) was to be found for 15 threads for the 1s and 0s array, which further explains why this thread count displayed the best performance for this array. However, utilisation took a steep dive when more than 15 threads were in use across all array types. Again for the 1s and 0s array, average CPU utilisation was 11.057 for 16 threads, dropping to 8.491 for 17 threads and remaining relatively stable at around 10.00–10.40 from 22 threads upwards. As noted above, the effect of scheduling from 17 threads upwards likely accounts for these discrepancies.

Number of CPUs utilised

Link together scalability investigation

See TODO

Max array sizes

Used average times etc. where could

Discuss issues with 1.0s boundary array as mentioned it in the correctness

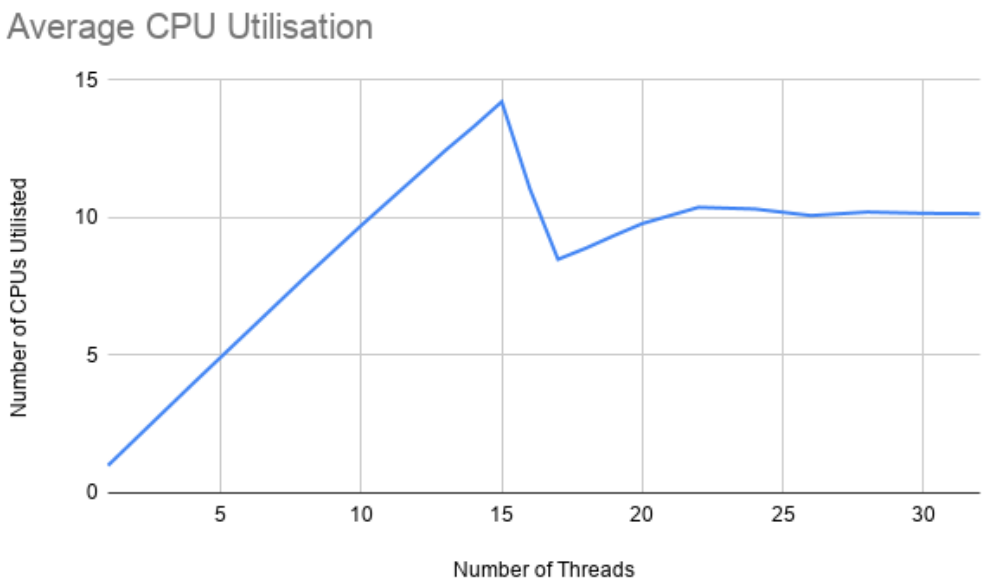

Figure 17: Average Number of CPUs Utilised (2200x2200) - 1s and 0s array testing section

4.1 Efficiency

Efficiency was a challenge to measure for this implementation, due to the constant interaction between serial and parallel elements of the program. However, an informed estimate could be provided using a wall-clock measure within the program itself. As well as measuring the total running time of the program, the major sequential elements of the program — namely that serial processing prior to thread creation, as well as those periods when threads were blocked by wait functions — could be timed, and hence the total parallel running time be deduced by the difference between the two.

There is a markedly different trend in how efficiency varies with array size when examining the two main array types mentioned (see Figure 18 and Figure 19). One notable characteristic, however, is the thread count with peak efficiency is also the thread count with peak speedup (see Speedup section below) and the lowest running time on a given dimension. For example, with the default array 13 threads provides the lowest running time at 550x550, as does 15 threads at 2400x2400 with a 1s and 0s array — both this specifications also provide peak efficiency and peak speedup in their class. Given efficiency is a measure of speedup per processor this follows from the theoretical definition.

Efficiency on the largest array size measured for the default array (550x550) was highest for a thread count of 13, reaching a peak of 38.5%, whilst for the 1s and 0s array the peak efficiency was 77.7% with 15 threads an array size of 2200x2200.

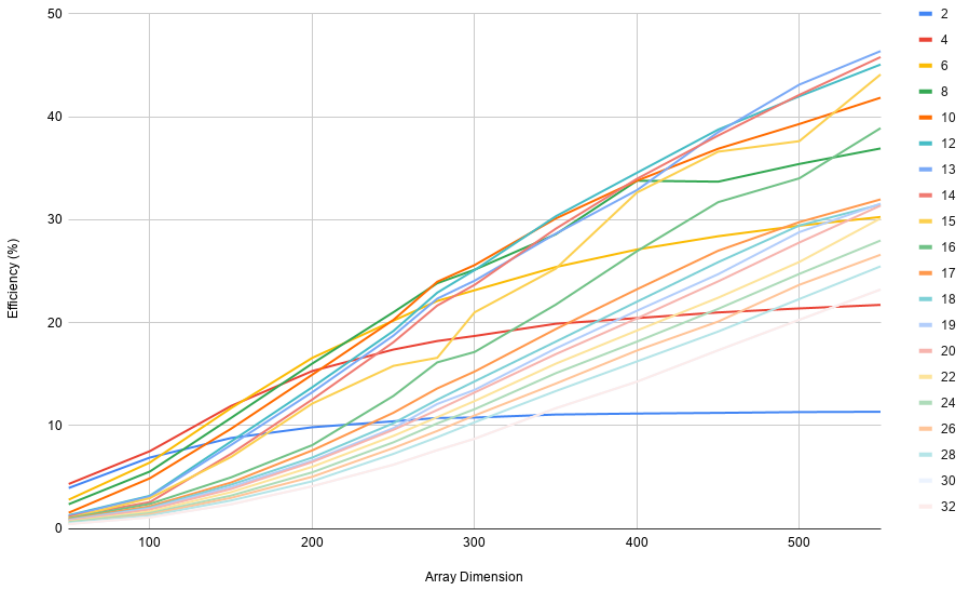

Figure 18: Efficiency vs array dimension for various thread counts

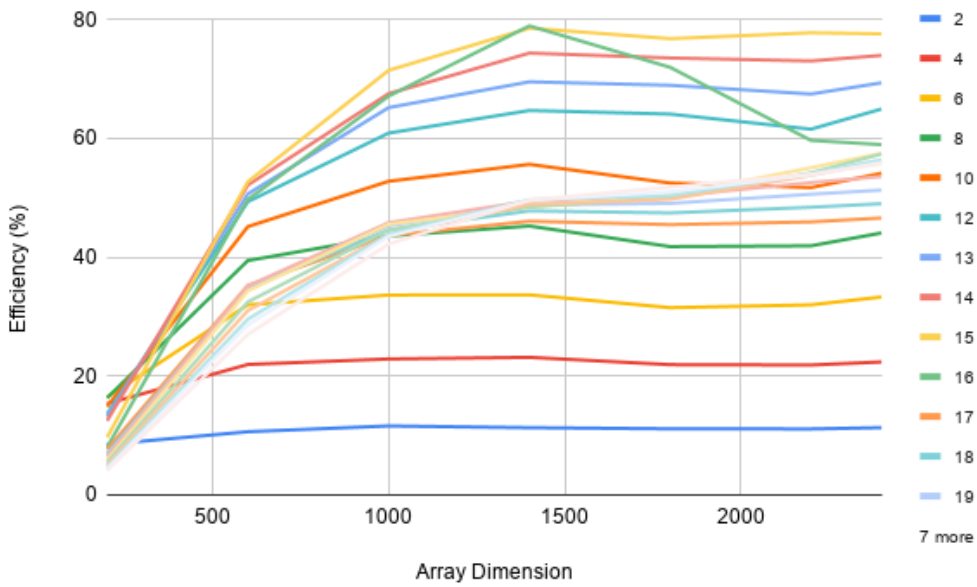

Figure 19: Efficiency vs array dimension for various thread counts — 1s and 0s array

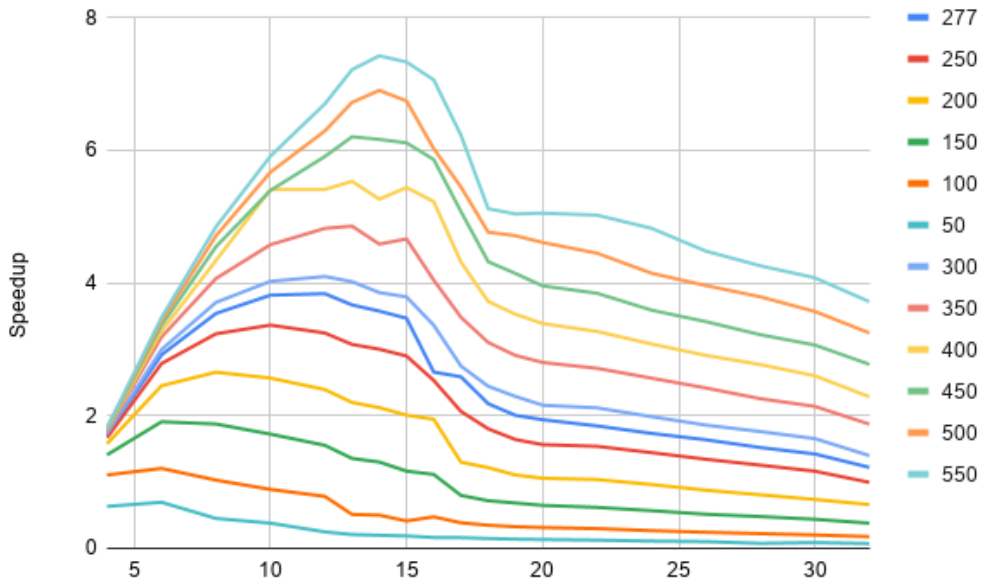

Figure 20: Speedup vs Thread Count for various array dimensions

4.2 Speedup

Speedup increases alongside thread count up to a maximum at 13 threads for the default array and 15 threads for the 1s and 0s array — the maximum speedup measured was 12.44 for an 2200x2200 array and 15 threads on the latter array type. The increase in speedup with thread count was expected, as overheads such as thread creation and serial elements of the program became relatively smaller in comparison to time spent processing in parallel. A speedup of 12.44 was also much greater than anticipated — relatively close to Amdahl's limit of 15.50 for this configuration (based off a serial running time of 0.099s and parallel running time of 45.73s), as well as an identical Gustafson's limit.

4.3 DELETE BEFORE SUBMISSION

If have time, get results for different array contents, and change above to reflect this.

A Code

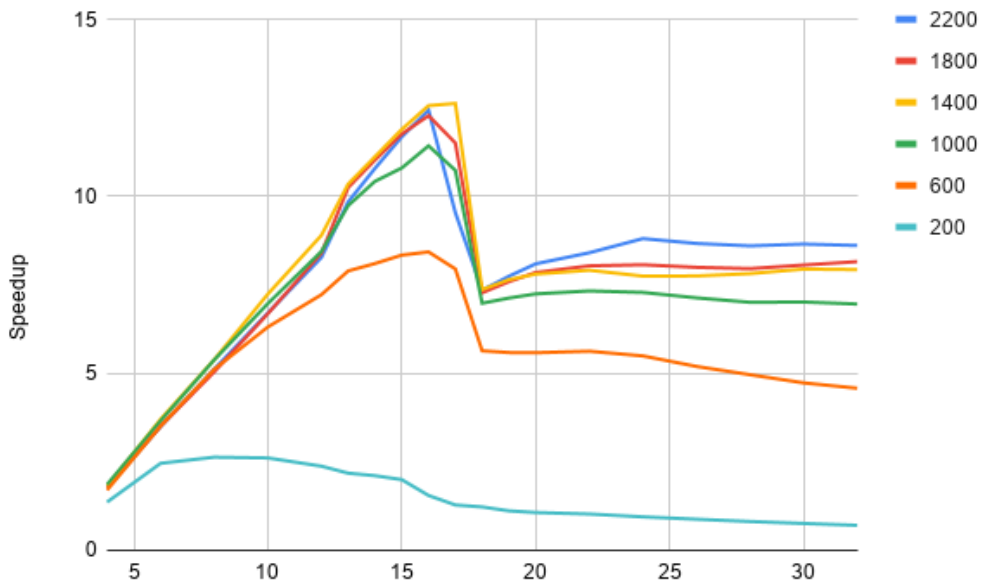

Figure 21: Speedup vs Thread Count for various array dimensions - 1s and 0s array